The following listing shows the annotated code that implements these changes inside the chat page.js component. We are listing only the parts that change since the rest of the code for the page remains the same.

Listing 3.8 Main application page

```
'use client';
import React from 'react';
import { Textarea } from '@/components/ui/textarea';
import ChatList from '@/components/chat/ChatList';
import useEnterSubmit from '@/hooks/use-enter-submit';
import useFocusOnSlashPress from '@/hooks/use-focus-on-slash-press';
// import { getAssistantResponse }
// from '@/lib/getAssistantResponse';

import { useChat } from 'ai/react';

const Chat = () => {
  const { formRef, onKeyDown } = useEnterSubmit();
  const inputRef = useFocusOnSlashPress();
  // const { messages, isLoading, handleSubmit, inputValue,
// setInputValue } = useChatFormSubmit(getAssistantResponse);
  const {
            messages,
            input,
            handleInputChange,
            handleSubmit, isLoading } = useChat({ api: '/api' });
 // ... the rest of the code remains as is
   }
```

The custom getAssistantResponse function is no longer needed since the useChat hook handles the API communication.

Imports the useChat hook from the ai/react package to handle the streaming of chat messages

Removes the custom useChatFormSubmit hook along with its associated state variables and functions

Uses the useChat hook to manage the chat state, including messages, input, handleInputChange, handleSubmit, and isLoading. Passes the API endpoint /api to the hook.

Now start the development server using the following command from the root of the project, and visit `localhost:3000`:

```
$  npm run dev -w ch03/chat-streaming
```

Streaming improves the user experience by delivering the assistant's response in real time, rather than waiting until the entire answer is generated. This makes conversations feel more natural and interactive. By replacing the custom hook with `useChat` from the `ai` package, we also reduce boilerplate code and gain built-in support for streaming updates.

Now you should see that the generated responses are being streamed sentence by sentence instead of updating all at once. Feel free to ask the AI to generate random long text of 300 to 500 words, for example, and compare the improvement in the user experience.